

File I/O and Strings

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

- File I/O
- Text Files
 - Character I/O
 - Formatted I/O
- Binary Files
- Pattern matching with scanf

File I/O

- Our good friend, `stdio.h`, also includes support for file I/O

```
#include <stdio.h>
```

- We just need to ask it to open a new file.

```
FILE *stream = fopen( "file-name.txt", "r" );
```

Path to the file you want.

Mode String (reading, writing, etc).

- This creates a new *stream* and returns a *file pointer* for working with it.
 - Or it returns **NULL on failure**.

File Mode

- When you open a file, the *mode string* says how you want to access it.

```
FILE *fp = fopen( "file-name.txt", "r" );
```



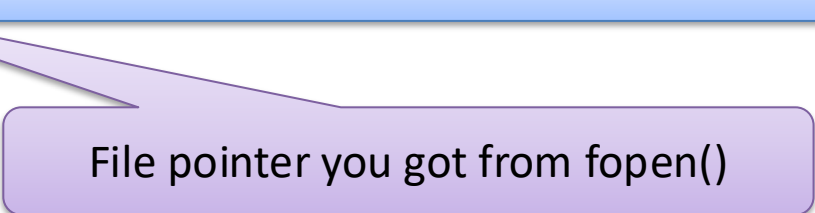
Mode String	Access Type
"r"	Open for reading
"w"	Open for writing (created and truncated)

More mode strings soon

Closing a File

- When you're done with a file ... you should close it.
- There's a function for that.

```
int fclose( FILE *stream );
```



File pointer you got from fopen()

- If you open files over and over but forget to close them, we call that a ***file leak***.
- That's going to be a problem for someone in the future ... probably you.

Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.

What you see in a text editor.

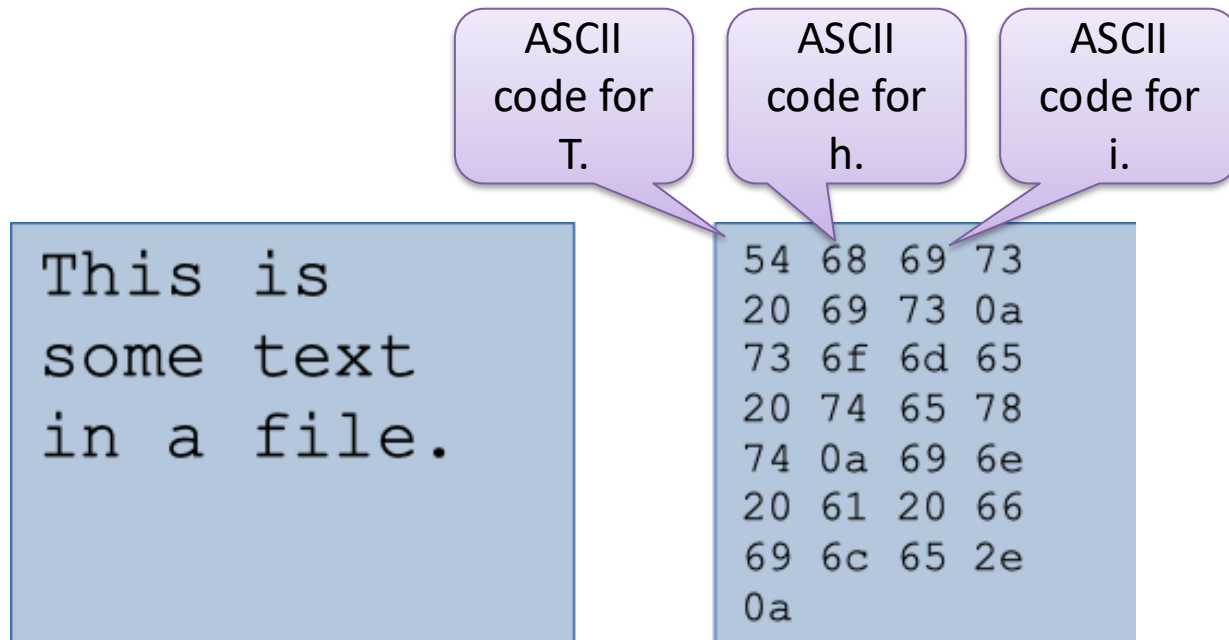
```
This is  
some text  
in a file.
```

What's really in the file.

```
54 68 69 73  
20 69 73 0a  
73 6f 6d 65  
20 74 65 78  
74 0a 69 6e  
20 61 20 66  
69 6c 65 2e  
0a
```

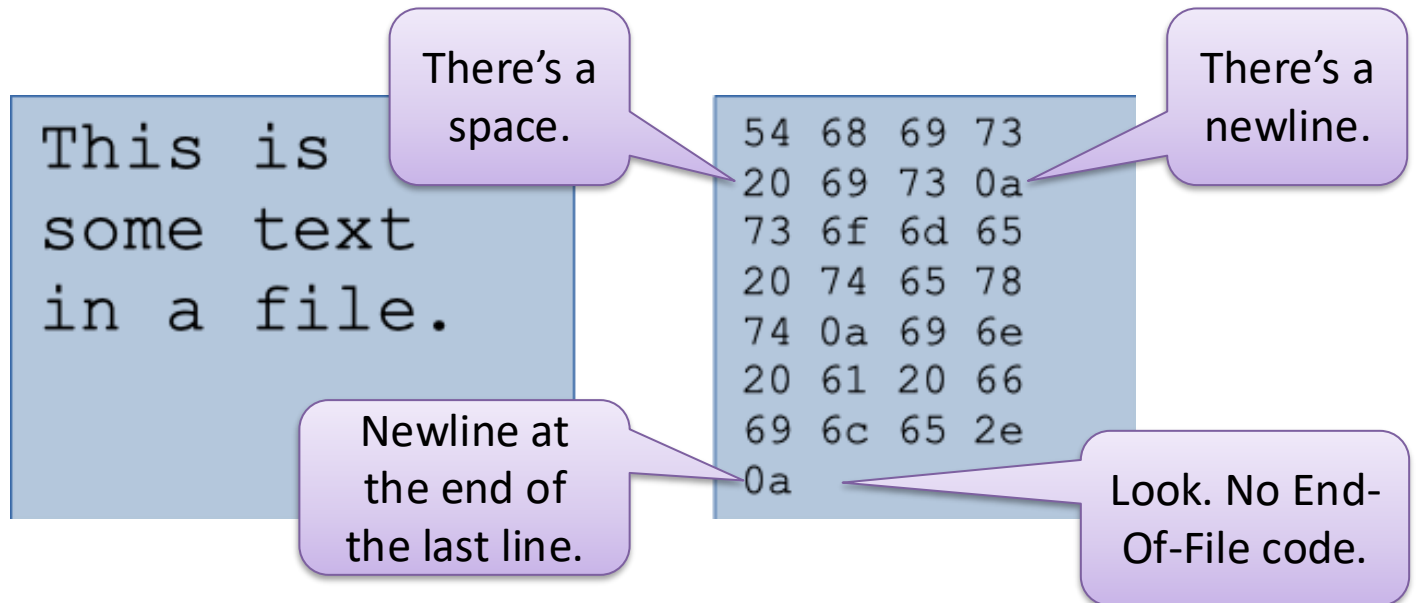
Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.



Text Files

- A file is just a sequence of bytes
- For an *ASCII Text file*, each byte is an ASCII character code.



Meet Hexdump

- The hexdump command can show you the exact sequence of bytes in a file.
 - It has options to show file contents in various formats.

Show numeric codes and characters.

File you want to see.

```
$ hexdump -C someFile.txt
00000000  54 68 69 73 20 69 73 0a 73 6f 6d 65 20 74 65 78 |This is.some tex|
00000010  74 0a 69 6e 20 61 20 66 69 6c 65 2e 0a          |t.in a file..|
```

Hexadecimal for each byte of the file.

Characters for each byte (if printable)

Wide Characters

- One-byte codes aren't sufficient for all character sets.
- C supports a wide character type, **wchar_t**
- In a file, a format like UTF-8 supports Unicode
 - Using 1-byte codes for ASCII characters
 - ... and multi-byte codes for other characters.

```
$ hexdump -C uni.txt
```

```
00000000  61 62 63 ce 91 ce 92 ce 93 0a      |abc.....|
```

One-byte codes
for a, b, c

Two-byte codes
for Greek Α, Β, Γ

Character I/O

- We have functions for character I/O on the terminal

```
int getchar(void) ;
```

```
int putchar(int ch) ;
```

- They read/write a single character
- Return EOF on failure.

- We have similar functions for working with any stream

```
int fgetc(FILE *stream) ;
```

I read a single char
from the given stream.

```
int fputc(int ch, FILE *stream) ;
```

I write a single char to
the given stream.

Character File I/O

- Copying a file character-by-character:

```
FILE *input = fopen( "input.txt", "r" );  
FILE *output = fopen( "output.txt", "w" );
```

Open a source and destination file.

```
int ccount = 0;  
int ch;
```

```
while ( ( ch = fgetc( input ) ) != EOF ) {  
    ccount += 1;  
    fputc( ch, output );  
}
```

Read characters until the end-of-file

Count each character and write it out.

```
fclose( input );  
fclose( output );
```

Close files when we're done.

```
printf( "Copied: %d\n", ccount );
```

Formatted I/O

- We have a version of `printf( )` for writing to any stream.

```
int fprintf(FILE *stream, const char format[], ...);
```

Stream you want
to write to.

The rest is just like
`printf`.

- We have a similar version of `scanf( )`

```
int fscanf(FILE *stream, const char format[], ...);
```

Same here, just one new
parameter at the start.

Formatted File I/O

- Copying all the words in a file?

```
FILE *input = fopen( "input.txt", "r" );  
if ( ! input )  
    ...
```

Open a file for reading

What if we fail?

```
FILE *output = fopen( "output.txt", "w" );  
if ( ! output )  
    ...
```

Same for output file

```
char word[ 11 ];  
while ( fscanf( input, "%10s", word ) == 1 )  
    fprintf( output, "%s\n", word );
```

Read words up to 10 chars

Write them to the output
file, one per line.

```
fclose( input );  
fclose( output );
```

All done with the files.

Leaking Files

- There's typically a limit on the number of open files you can have
 - That's why leaking files is bad
- I have a test for this: `openTest.c`
 - It opens files over and over until it can't create any more
 - Let's give it a try

Three Streams for Free

- Your program gets three streams at startup, you've been working with them all semester
 - **stdin** : standard input (from the terminal by default)
 - **stdout** : standard output (to the terminal by default)
 - **stderr** : standard error (also to the terminal by default)
- These are all global variables declared in **stdio.h**
- This is secretly why openTest.c could open 1021 files.
 - 3 files open at startup + 1021 more files opened = 1024 files.

Using the Standard Streams

- With names for our standard streams
 - This gives us alternative ways to do terminal I/O.

```
fscanf( stdin, ... );
```



```
scanf( ... );
```

```
fprintf( stdout, ... );
```



```
printf( ... );
```

```
fprintf( stderr, ... );
```

We didn't need the first two,
but here's something new.

- Printing to stderr is a standard way of reporting, well, errors.
 - Why? We can redirect it differently from stdout.

Using stderr

- We can use stderr for any error reports

```
FILE *fp = fopen( "numbers.txt", "r" );
if ( !fp ) {
    fprintf( stderr, "Can't open input file\n" );
    exit( EXIT_FAILURE );
}

int len = 0;
if ( fscanf( input, "%d", &len ) != 1 ) {
    fprintf( stderr, "Can't read input length\n" );
    exit( EXIT_FAILURE );
}

. . .
```

Redirecting Streams

- From the shell, we can choose to redirect streams differently.

I redirect standard output.

I redirect standard error.

```
$ ./program > output.txt 2> error.txt
```

- Or the same

Send standard error to the same file as standard output.

```
$ ./program > everything.txt 2>&1
```

- This syntax is dependent on your shell, but this is what bash uses.

Text vs Binary Mode

- A streams can operate in two modes:
 - ***Text mode*** : tries to hide some details like the line termination sequence.
 - ...

Text File I/O

- An input stream may convert some input as you read ...
- ... and as you write.

Text in the file.

```
This is  
a text file  
with windows  
line  
termination.
```

Bytes that are really there
(e.g., on Windows)

54	68	69	73	20	69	73	0d
0a	61	20	74	65	78	74	20
66	69	6c	65	0d	0a	77	69
74	68	20	77	69	6e	64	6f
77	73	0d	0a	6c	69	6e	65
0d	0a	74	65	72	6d	69	6e
61	74	69	6f	6e	2e	0d	0a

What you see as you read.

54	68	69	73	20	69	73	0a
61	20	74	65	78	74	20	66
69	6c	65	0a	77	69	74	68
20	77	69	6e	64	6f	77	73
0a	6c	69	6e	65	0a	74	65
72	6d	69	6e	61	74	69	6f
6e	2e	0a					

Text I/O

- Text mode can lie to you about what's in the file

```
FILE *input = fopen( "input.txt", "r" );
FILE *output = fopen( "output.txt", "w" );

int ccount = 0;
int ch;
while ( ( ch = fgetc( input ) ) != EOF ) {
    ccount += 1;
    fputc( ch, output );
}

fclose( input );
fclose( output );

printf( "Copied: %d\n", ccount );
```

54	68	69	73	20	69	73	0d
0a	61	20	74	65	78	74	20
66	69	6c	65	0d	0a	77	69
74	68	20	77	69	6e	64	6f
77	73	0d	0a	6c	69	6e	65
0d	0a	74	65	72	6d	69	6e
61	74	69	6f	6e	2e	0d	0a

run on
Windows

Copied: 51

Binary I/O

- A streams can operate in two modes:
 - **Text mode** : tries to hide some details like the line termination sequence.
 - **Binary mode** : shows you every character/byte exactly.
- Every stream you've used so far has been in text mode.
- On the common platform it doesn't matter.
 - They're exactly the same.
 - But, on other systems they may not be.

Meet More Mode Strings

- In the mode string, you can choose text or binary mode ... and other things.

```
FILE *fopen(const char *path, const char *mode);
```

Mode String	Access Type
r	Open for reading
w	Open for writing (created and truncated)
rb	Open a binary file for reading
wb	Open a binary file for writing

More combinations we could make

Binary I/O

- Binary mode will show you exactly what's in the file.

```
FILE *input = fopen( "input.txt", "rb" );
FILE *output = fopen( "output.txt", "wb" );

int ccount = 0;
int ch;
while ( ( ch = fgetc( input ) ) != EOF ) {
    ccount += 1;
    fputc( ch, output );
}

fclose( input );
fclose( output );

printf( "Copied: %d\n", ccount );
```

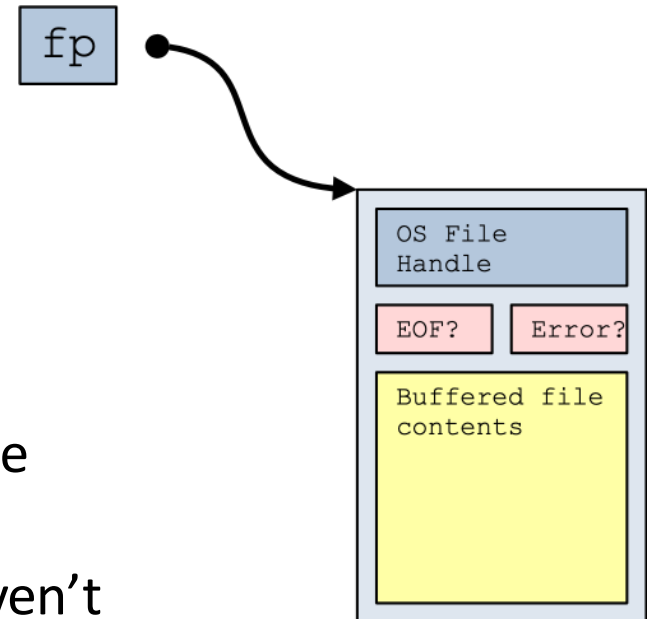
54	68	69	73	20	69	73	0d
0a	61	20	74	65	78	74	20
66	69	6c	65	0d	0a	77	69
74	68	20	77	69	6e	64	6f
77	73	0d	0a	6c	69	6e	65
0d	0a	74	65	72	6d	69	6e
61	74	69	6f	6e	2e	0d	0a

run on
Windows

Copied: 56

Buffering

- For each open file, there's a record of information about that file.
- That's what a FILE * points to.
- It contains:
 - System specific information for accessing the file.
 - Flags for the state of the file
 - A big **buffer** containing
 - Data you've written that hasn't gone to the file yet.
 - Data read from the file that you haven't seen yet.



Unwritten Buffered Data

- What if we don't get to close a file?

```
FILE *input = fopen( "input.txt", "r" );  
FILE *output = fopen( "output.txt", "w" );
```

```
char word[ 11 ];  
while ( fscanf( input, "%10s", word ) == 1 )  
    fprintf( output, "%s\n", word );
```

```
int x[ 10 ];  
printf( "%d\n", x[ 9999999999 ] );
```

```
fclose( input );  
fclose( output );
```

Ouch. This will probably crash our program.

We won't get a chance to close the file.

Buffered data won't get written out.

Flushing the Buffer

- Normally, standard output flushes automatically ... when it sees a newline

```
printf( "You can't see me " );  
sleep( 5 );  
printf( "... until you write a newline\n" );
```

I see this mistake all the time
with debug output.

- Closing a file will flush the buffer.
- Sometimes you want your output **right now!**

```
int fflush( FILE *stream );
```

- Buffered output is written out immediately

Flushing the Buffer

- You can take control of this with fflush()

```
printf( "But flushing writes output immediately" );  
fflush( stdout );  
sleep( 5 );  
  
printf( "... even without a newline.\n" );
```

- Flushing the buffer requires help from the Operating System.
- Can slow your program down if you do it too often.

Take Backsies

- An input stream will let you return **one input character** to the buffer

```
int ungetc(int c, FILE *stream);
```

- Pushes character *c* back onto the given stream
- It even works with **stdin**
- You can't push back an EOF, only a real character.

Taking Another Look

- Normally, files are accessed **sequentially**
 - After you read/write a byte you automatically get the next byte
 - Buffering done inside the library promotes sequential access performance.
- But, you you can go back to the start of a file.
 - To, re-read or re-write it from the start.

```
int rewind( FILE *stream );
```

Jumping Around

- There's a more general function to seek to anywhere.

What stream?

Where to?

With respect to what?

```
int fseek(FILE *stream, long offset, int whence);
```

- The whence parameter tells you with-respect to what?
 - SEEK_SET : just go to this offset
 - SEEK_CUR : offset is with-respect-to the current position
 - SEEK_END : offset is from the end of the file.
- Works for binary streams, limited functionality for text

```
fseek( stream, -1, SEEK_CUR ); // Backup one byte
fseek( stream, 0, SEEK_END ); // Go to end of file
fseek( stream, p, SEEK_SET ); // Go to position p
```

Checking Your Location

- How do you know where to seek to in a file?
- You could count characters as you read.
 - But, that sounds like some extra work.
 - And, it may not work very well for text files ...
 - ... you may not be seeing the exact sequence of bytes in the file.
- But, a stream can tell you your current location.

```
long ftell( FILE *stream );
```

- You can fseek back to this location to return and read from there again.

String Parsing Options

- The %s format specifier reads
 - space-delimited words ... of arbitrary length.

```
char str[ 11 ];  
scanf( "%s", str );
```

Potential buffer overflow

- You know you can prevent buffer overflow by providing a field width.

```
char str[ 11 ];  
scanf( "%10s", str );
```

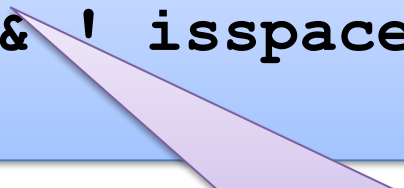
Read at most a 10-character word.

- How can you tell if the input word was too long?

Detecting Excessive Input

- Option 1 : check the next character.
 - If it's still a word character, scanf must have stopped reading inside a word.

```
char str[ 11 ];  
if ( scanf( "%10s", str ) == 1 ) {  
    int ch = getchar();  
    if ( ch != EOF && ! isspace( ch )  
        // bad input
```



Peek ahead at the next character.

Detecting Excessive Input

- Option 2 : permit one extra character in the input.
 - This is enough to let you detect when the input was too long.
 - (without actually reading the whole thing)

```
char str[ 12 ];  
if ( scanf( "%11s", str ) == 1 ) {  
    if ( strlen( str ) > 10 )  
        // bad input
```

One extra character.

Was the input too long?

String Parsing Options

- The %s format specifier reads space-delimited words.
- scanf() also supports a subset of regular expression syntax.
 - Giving you more control of the string you read.
 - Using square brackets we can define a *character class*, a set of exactly what characters to match.

```
scanf( "[%abc]", str );
```

Match a sequence of one or more a, b and c characters.

Doesn't skip spaces.

Character Class Examples

```
scanf( "[%abc]", str );
```

bcabbxaccb

This will return zero matches.

zero

```
scanf( "[% 0123456789]", str );
```

123 45w xyz 789

Not (Quite) Regular Expressions

- Regular expressions : a common pattern language for matching strings
- `scanf()/fscanf()` support a similar syntax
 - but, it's not exactly the same
 - generally not as powerful / expressive

String Parsing Options

- Inside the brackets, we can specify ranges of characters.

```
scanf( "%[A-Z]", str );
```

Match any capital letter.

- We can combine ranges and individual characters.

```
scanf( "%[a-zA-Z.]", str );
```

Match lower-case or capital letters or dot.

abc.XYZ.123

String Parsing Options

- You can negate a character class by starting it with a [^]

```
scanf( "%[^\\n]", str );
```

Match anything but a newline.

```
scanf( "%[^a-zA-Z]", str );
```

Anything but a letter.

123-456-abcd

String Parsing Options

- Using `*`, we can tell `scanf` to match a format specifier, but not save it:

```
scanf( "%*s%s", str );
```

Skip spaces, then save the next word in `str`.

Match a word (but don't store it)

- Really, this works with any format specifier.

```
scanf( "%*d%d", &val );
```

```
scanf( "%*[a-z]%20[A-Z]", str );
```

Skipping Whitespace

- Any whitespace in a format string ... matches any amount of whitespace.

```
scanf( "%c", &ch );
```

I'll read the next character, even if it's whitespace

```
scanf( " %c", &ch );
```

I'll skip all whitespace, then read a character

```
scanf( "%[0-9]", str );
```

I'll read just digits.

```
scanf( " %[0-9]", str );
```

I'll skip whitespace, then read digits.

Skipping Whitespace

- Be careful, stray whitespace can give seemingly strange behavior.

```
scanf( "%d%d\n", &a, &b );
```

OK. You want me to skip whitespace after reading these ints.

I better not return until I'm sure I have it all.

Matching Literal Characters

- Literal characters will match copies of themselves.

```
scanf( "%d+%d", &a, &b );
```

I'll match the +, but I won't skip spaces before it.

```
25+72
```

```
25 + 72
```

```
scanf( "%d +%d", &a, &b );
```

But I will.

```
scanf( "%d %%", &a );
```

Need to match a literal %? We got that.

Formatted Input from a String

- The standard library as functions to parse input from a file.
- There's also a function for parsing the contents of a string

```
int sscanf(const char str[], char const format[], ...);
```

- Parse the contents of str according to the given format string.
 - Assign remaining arguments based on the given conversion specifiers
 - Return the number of conversion specifiers matched.
- Kind of like wrapping a Scanner around a string in Java.
- These are declared in stdio.h

Formatted Input from a String

- Example, parse an int and a float from the contents of a buffer:

```
char buffer[] = "25 31.2";  
int iVar;  
float fVar;  
sscanf( buffer, "%d%f", &iVar, &fVar );
```

- This function is why I almost never use atoi(), atof() ...
 - sscanf() can do the job of all of them.
 - And it gives you a more useful success/failure report.

```
int val = atoi( str );
```

I return zero on an error.

```
if ( sscanf(str, "%d", &val) != 1 )  
    oops;
```

I save a value and return a success/value status.

Formatted Output to a String

- We have similar functions for going the other way:

```
int sprintf(char str[], char const format[], ...);
```

```
int snprintf(char str[], size_t n, char const format[], ...);
```

- Like printf() but with output going to the given string.
- snprintf() writes at most n bytes (including a null terminator at the end)

There's a lot of inconsistency here.

- Example:

```
char buffer[ 10 ];  
snprintf( buffer, sizeof(buffer), "%s", "abcdefghijklmnop" );
```

Copies just the first 9 characters.